

momu-agent-framework

参考 HelloAgents 实现的智能体框架，提供 Agent、工具系统、记忆系统、RAG、上下文工程与 OpenAI 兼容 LLM 接入能力。



特性

- **全异步设计**: `Agent.run()`、工具执行主路径、工具链与上下文构建均为 `async`
- **Agent 体系**: 抽象基类 `Agent` + 四种开箱即用 Agent: `SimpleAgent`、`ReActAgent`、`PlanSolveAgent`、`ReflectionAgent`
- **工具系统**: `ToolRegistry` 统一管理工具，支持 `Tool` 对象、同步函数和异步函数注册
- **工具链与并发**: `ToolChain` / `ToolChainManager` 顺序编排，`ToolExecutor` 并发执行
- **流式输出**: `SimpleAgent.stream_run()` 支持逐段异步输出
- **内置工具**: `CalculatorTool`、`SearchTool`、`RAGTool`、`MemoryTool`、`NoteTool`、`TerminalTool`
- **上下文工程**: `ContextBuilder` 实现 GSSC (Gather-Select-Structure-Compress) 流程
- **记忆系统**: `WorkingMemory`、`EpisodicMemory`、`SemanticMemory` 统一管理
- **RAG 能力**: 支持文本/文档入库、检索与基于上下文问答
- **OpenAI 兼容**: `LLM` 可接入任何兼容 OpenAI 接口的模型服务



安装

需要 Python 3.11+, 使用 [uv](#) 管理依赖。

基础功能

```
uv sync
```

按需安装扩展

```
uv sync --extra search
uv sync --extra memory
uv sync --extra rag
```

安装全部扩展

```
uv sync --all-extras
```



配置

复制 `.env.example` 为 `.env` 并填写配置:

```
cp .env.example .env
```

必填项:

```
LLM_MODEL_ID=qwen-turbo
LLM_API_KEY=sk-your-api-key-here
LLM_BASE_URL=https://dashscope.aliyuncs.com/compatible-mode/v1
```

常用可选项: `TEMPERATURE`、`MAX_TOKENS`、`TIMEOUT`、`MAX_HISTORY_LENGTH`、`LOG_LEVEL`、`TAVILY_API_KEY`、`SERPAPI_API_KEY`、`EMBED_MODEL_NAME`。

💡 快速上手

```
import asyncio

from momu_agent.agents import SimpleAgent
from momu_agent.core.llm import LLM
from momu_agent.utils.config import Config
from momu_agent.utils.logger import setup_logger

def build_llm(config: Config) -> LLM:
    return LLM(
        model=config.model_id,
        api_key=config.api_key,
        base_url=config.base_url,
        temperature=config.temperature,
        max_tokens=config.max_tokens,
        timeout=int(config.timeout),
    )

async def main() -> None:
    config = Config.from_env()
    setup_logger(level=config.log_level)

    agent = SimpleAgent(
        name="Support",
        llm=build_llm(config),
        system_prompt="你是一名 Python 技术支持助手，回答简洁准确。",
        max_history_length=config.max_history_length,
    )

    print(await agent.run("什么是 GIL? "))
    async for chunk in agent.stream_run("介绍 Python 异步编程的最佳实践。"):
        print(chunk, end="", flush=True)
```

```
asyncio.run(main())
```

运行示例

```
uv run python examples/simple_agent_demo.py
uv run python examples/react_agent_demo.py
uv run python examples/plan_solve_agent_demo.py
uv run python examples/reflection_agent_demo.py
uv run python examples/memory_tool_demo.py
uv run python examples/rag_tool_demo.py
uv run python examples/context_aware_agent_demo.py
```

目录结构

```
momu_agent/
├─ core/           # Agent 抽象、LLM、消息、异常
├─ tools/          # Tool 抽象、Registry、Chain、Executor、内置工具
│   └─ builtin/    # calculator/search/rag/memory/note/terminal
├─ agents/         # Simple/ReAct/PlanSolve/Reflection
│   └─ parser/     # 工具调用解析器
├─ context/        # ContextBuilder (GSSC)
├─ memory/         # working/episodic/semantic + manager
├─ storage/        # document/vector/graph 存储
├─ rag/            # 文档处理与检索管线
└─ utils/          # config/logger/embedding 等辅助模块
```

开发

```
uv run pytest
uv run pytest tests/context/test_builder.py
uv run ruff check .
uv run ruff format --check .
uv run ruff check --fix .
uv run ruff format .
```

CI 在 `main` 的 push/PR 上运行:

- `lint`: Python 3.13 + Ruff 检查
- `test`: Python 3.11 / 3.12 / 3.13 运行 pytest (含 sentence-transformers 缓存与 CPU 版 torch 安装)

参考

本项目参考了 [HelloAgents](#) 的设计思想与实现架构。

致谢

感谢 Datawhale 提供的 [《HelloAgents》](#) 教程。